

## **CODE:**

```
// =====  
  
// 1. SINGLE INHERITANCE  
  
// =====  
  
// Parent class  
class Fruit {  
    void eat() {  
        System.out.println("Fruit is eaten");  
    }  
}  
  
// Apple inherits from only one parent class: Fruit  
class Apple extends Fruit {  
    void bite() {  
        System.out.println("An apple a day keeps the doctor away");  
    }  
}  
  
// =====  
  
// 2. MULTILEVEL INHERITANCE  
  
// =====  
  
// Fruit -> Apple -> Orange  
// Orange inherits properties from Apple and indirectly from Fruit.  
  
class Orange extends Apple {
```

```

void juice() {
    System.out.println("Orange gives juice");
}
}

// =====

// 3. HIERARCHICAL INHERITANCE

// =====

// Multiple child classes inherit from the same parent class.
// Here, Lion and Cat both inherit from Animal.

class Watermelon extends Fruit {
    void colour() {
        System.out.println("Watermelon is green outside and red inside");
    }
}

// =====

// 4. MULTIPLE INHERITANCE USING INTERFACES

// =====

// Java does NOT allow:
// class C extends A, B
//
// This is because it can create ambiguity (Diamond Problem).
//
// Instead, Java allows a class to implement multiple interfaces.

```

```
interface Father {  
    void fatherProperty();  
}
```

```
interface Mother {  
    void motherProperty();  
}
```

```
// Child implements two interfaces.
```

```
// Therefore, it gets features from both Father and Mother.
```

```
class Child implements Father, Mother {  
    public void fatherProperty() {  
        System.out.println("Property inherited from Father");  
    }  
    public void motherProperty() {  
        System.out.println("Property inherited from Mother");  
    }  
}
```

```
// =====
```

```
// MAIN CLASS
```

```
// =====
```

```
public class Main {  
    public static void main(String[] args) {
```

```
// ----- SINGLE INHERITANCE -----
```

```
Apple a = new Apple();
```

```
// eat() is inherited from Fruit
```

```
a.eat();
```

```
// bite() belongs to Apple
```

```
a.bite();
```

```
System.out.println("-----");
```

```
// ----- MULTILEVEL INHERITANCE -----
```

```
Orange o = new Orange();
```

```
// eat() comes from Fruit
```

```
o.eat();
```

```
// juice() comes from Orange
```

```
o.juice();
```

```
System.out.println("-----");
```

```
// ----- HIERARCHICAL INHERITANCE -----
```

```
Watermelon w = new Watermelon();
```

```
// eat() is inherited from Fruit
```

```
w.eat();
```

```
// colour() belongs to Watermelon
```

```
w.colour();
```

```
System.out.println("-----");

// ----- MULTIPLE INHERITANCE -----

Child ch = new Child();

// Child gets methods from both interfaces
ch.fatherProperty();
ch.motherProperty();
}
}
```

## **Explanation:**

### 1.Single Inheritance

---

```
class Fruit {
void eat()
{
System.out.println("Fruit is eaten");
}}
class Apple extends Fruit {
void bite() {
System.out.println("An apple a day keeps the doctor away");
}
}
```

---

- Fruit is the parent (super) class.
- Apple is the child (sub) class.
- The keyword **extends** makes Apple inherit Fruit.
- Therefore, an Apple object can use both eat() and its own bite() method.
  - In main(): Apple a = new Apple();
  - a.eat(); // inherited from Fruit
  - a.bite(); // belongs to Apple
  - Output: Fruit is eaten An apple a day keeps the doctor away.

## 2.Multilevel Inheritance

---

```
class Orange extends Apple {  
    void juice() {  
        System.out.println("Orange gives juice");  
    }  
}
```

---

- Orange directly inherits Apple. Since Apple already inherits Fruit.
- Orange can also access the inherited eat() method from Fruit.
- In main(): Orange o = new Orange();
- o.eat(); // comes from Fruit.
- o.juice(); // belongs to Orange
- Orange also inherits bite() from Apple, even though this particular main() code does not call it. This is called multilevel inheritance because inheritance occurs through more than one level.

## 3.Hierarchical Inheritance

---

```
class Watermelon extends Fruit {  
    void colour() {  
        System.out.println("Watermelon is green outside and red inside");  
    }  
}
```

---

- Fruit is the common parent of both Apple and Watermelon.
- Watermelon inherits eat() from Fruit and has its own colour() method.
- In main(): Watermelon w = new Watermelon();
- w.eat(); // inherited from Fruit
- w.colour(); // belongs to Watermelon Because multiple child classes inherit from the same parent, this is hierarchical **inheritance**.

## 4. Multiple Inheritance Using Interfaces

---

```
interface Father {  
    void fatherProperty();  
}  
  
interface Mother {  
    void motherProperty();  
}  
  
class Child implements Father, Mother {  
    public void fatherProperty() {  
        System.out.println("Property inherited from Father");  
    }  
}
```

```
public void motherProperty() {  
    System.out.println("Property inherited from Mother");  
}  
}
```

---

- Java does not allow a class to extend two classes at the same time: class Child extends Father, Mother // NOT allowed.
- Instead, Java allows a class to implement multiple interfaces. implements Father, Mother means Child agrees to provide implementations for the methods declared in both interfaces.
- In main(): Child ch = new Child();
- ch.fatherProperty(); ch.motherProperty(); The Child object can therefore use methods from both interfaces.
- This is commonly called multiple inheritance through interfaces. The interfaces in this example contain abstract method declarations, and Child provides their implementations.

## 5.Main Method and Object Creation

---

```
public class Main {  
    public static void main(String[] args) {  
        Apple a = new Apple();  
        Orange o = new Orange();  
        Watermelon w = new Watermelon();  
        Child ch = new Child();  
    }  
}
```

---

- The main() method is the starting point of the Java program.
- Each statement creates an object:
- Apple a = new Apple(); creates an Apple object.
- Orange o = new Orange(); creates an Orange object.
- Watermelon w = new Watermelon(); creates a Watermelon object.
- Child ch = new Child(); creates a Child object.
- The object is used to call the methods available through its class and inherited classes/interfaces.

## 6. Separator Lines

-----

```
System.out.println("-----");
```

-----

- They print a horizontal line so that the output of each inheritance example is easier to distinguish.

## 7. Expected Output

Fruit is eaten

An apple a day keeps the doctor away

-----

Fruit is eaten

Orange gives juice

-----

Fruit is eaten

Watermelon is green outside and red inside

-----

Property inherited from Father

Property inherited from Mother

The first section demonstrates single inheritance, the second demonstrates multilevel inheritance, the third demonstrates hierarchical inheritance, and the last demonstrates multiple inheritance using interfaces.

#### Output

```
Fruit is eaten
An apple a day keeps the doctor away
-----
Fruit is eaten
Orange gives juice
-----|-----
Fruit is eaten
Watermelon is green outside and red inside
-----
Property inherited from Father
Property inherited from Mother
```

```
=== Code Execution Successful ===
```